VeriVision AI

Product Documentation — AI-Generated Image Detection with Visual Explanations

VeriVision AI is a standalone professional web application that estimates whether an uploaded image is likely real or AI-generated. It combines four fine-tuned convolutional detectors in a logit-space ensemble, explains each decision with Grad-CAM heatmaps, and re-tests every upload under common distortions to report a robustness consistency score. All inference runs locally in a Flask application — no external APIs, no cloud uploads.

1. Product overview

VeriVision AI targets photographers, editors, moderators and any user who needs a fast, honest first opinion on image authenticity. The home page offers drag-and-drop upload; the result page shows the verdict (REAL / AI-GENERATED), confidence, per-model votes, a Grad-CAM explanation with branch selector, and a robustness check. A Model Insights page surfaces the measured evaluation metrics loaded from saved artifacts.

2. Problem being addressed

Modern generative models produce images that are difficult to distinguish by eye. Manual inspection does not scale and gives no calibrated confidence. VeriVision AI automates the first pass: a learned detector scores the image, visualizes its evidence, and reports how stable the verdict is under realistic degradations such as JPEG recompression and resizing. Results are explicitly framed as AI-based estimates, never as forensic proof.

3. Key features

Four-model heterogeneous ensemble; logit-space weighted fusion with disagreement gating; Grad-CAM explanations from real gradients (EfficientNet / ResNet50 / MobileNetV3 views); robustness consistency check (JPEG, resize, blur); Model Insights dashboard fed by real evaluation files; local Flask deployment bound to 127.0.0.1 with CPU/GPU auto-detect; one-click Windows launcher (run.bat) and Docker support.

4. System architecture

Browser → Flask (app/routes.py) → preprocessing validation (app/preprocessing.py) → per-route inference with Route-B flip TTA (app/inference.py) → logit fusion + gating (app/ensemble.py) → Grad-CAM overlays (app/explainability.py) → result page. Models load once at startup under torch.no_grad(); temporary files are cleaned up after each request. Training and evaluation live in scripts/ with shared logic in scripts/common.py.

5. CNN models used

Route A1: EfficientNet-B0, 224×224, CrossEntropy, normal augmentation. Route A2: EfficientNet-B0, 224×224, CrossEntropy, distortion-hardened augmentation (blur, noise, JPEG simulation, resampling, color shifts). Route B: ResNet50, 320×320, CrossEntropy, moderate augmentation, horizontal-flip test-time averaging to preserve fine-grained evidence. Route C: MobileNetV3-Large, 224×224, focal loss

(gamma=2), backbone mostly frozen — lightweight architectural diversity. Upper layers plus classifier heads are fine-tuned with AdamW, cosine schedule and early stopping on validation F1. Class mapping is 0 = REAL, 1 = AI_GENERATED everywhere.

6. Dataset and preprocessing

Primary dataset: CIFAKE — Real and AI-Generated Synthetic Images (via Kaggle), organized by scripts/prepare_dataset.py into deterministic stratified 70/15/15 splits at data/{train,val,test}/{real,fake}. Splitting precedes augmentation; augmentation applies to training only. Uploads are validated by extension (PNG/JPG/WEBP), size limit (10 MB default), successful RGB decoding and ImageNet normalization; corrupt files are rejected gracefully. In this build the Kaggle download was unreachable, so training used clearly-labelled local synthetic demo data with the identical schema — demo metrics must not be read as real-world performance.

7. Ensemble and gating strategy

Each route emits raw binary logits; Route B averages original and flipped logits. Fusion computes fused_logit = Σ(weight_i × logit_i) with default weights A1 0.35 / A2 0.15 / B 0.25 / C 0.25 (configurable in app/config.py), then softmax. Gate 1 excludes Route B when it strongly disagrees (margin ≥ tau_b = 2.5) with at least three other branch directions, renormalizing the rest. Gate 2 applies a small correction (0.75) toward a confident B+C consensus when it contradicts the fused verdict. Both rules are deterministic and covered by unit tests; parameters are engineering defaults to be tuned on validation data.

8. Grad-CAM explainability

Heatmaps come from actual model gradients via the standard pytorch-grad-cam package. Target layers: EfficientNet-B0 final convolutional block, ResNet50 layer4[-1], MobileNetV3-Large final feature convolution. The predicted class drives backpropagation; the resulting map is overlaid on the uploaded image. The UI selector switches between EfficientNet, ResNet50 and MobileNetV3 views, and the interface states that highlighted regions are an explanation aid, not proof of authenticity.

9. Web application workflow

GET / renders the upload page; POST /predict validates, runs the ensemble, generates three Grad-CAM overlays and four robustness variants, then renders the result page; POST /api/predict returns the same verdict as JSON; GET /insights renders saved metrics; GET /health reports model readiness. Missing artifacts produce a clear 503 message instead of a crash; oversized, unsupported or corrupt uploads return friendly 400 pages.

10. Evaluation methodology

scripts/evaluate.py scores every branch and the fused ensemble on the held-out test split (accuracy, precision, recall, F1, ROC-AUC, balanced accuracy, confusion matrix) and replays the ensemble on degraded copies (JPEG quality 60, 2× resampling, Gaussian blur, Gaussian noise), reporting clean vs degraded performance separately. Outputs land in artifacts/metrics/ as model_metrics.json, ensemble_metrics.json, robustness_metrics.json and confusion_matrix.png; the UI and this document read those files directly.

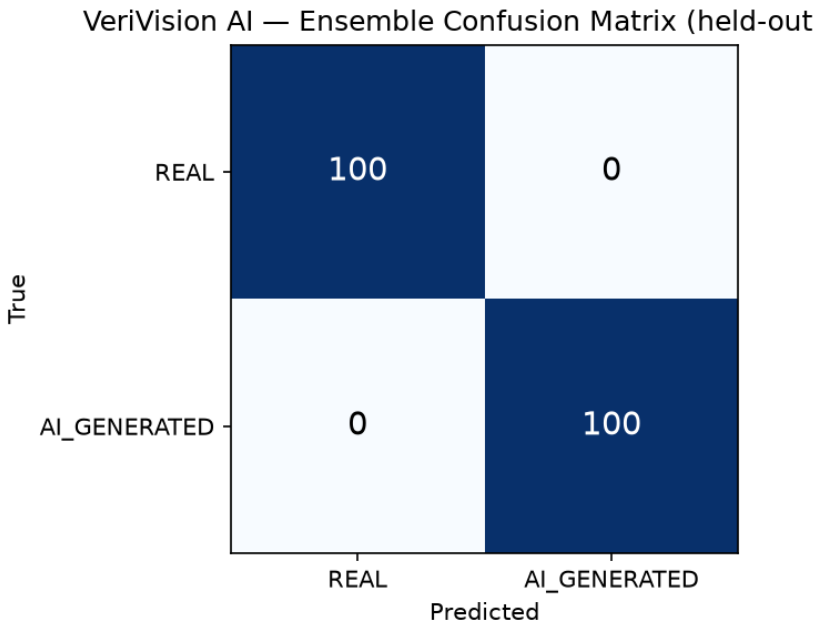
11. Actual measured results

Held-out test: 200 images · dataset tag: synthetic-demo-fallback · seed 42.

Metric	Ensemble value
Accuracy	100.00%
Precision	100.00%
Recall	100.00%
F1-score	100.00%
ROC-AUC	100.00%
Balanced accuracy	100.00%

Model	Accuracy	F1	ROC-AUC
efficientnet_a1	100.00%	100.00%	100.00%
efficientnet_a2	100.00%	100.00%	100.00%
resnet50_b	100.00%	100.00%	100.00%
mobilenetv3_c	100.00%	100.00%	100.00%
ensemble	100.00%	100.00%	100.00%

Ensemble confusion matrix (held-out test):



Because this build trained on synthetic demo data, the near-perfect clean scores reflect demo separability, not real-world capability. The degraded rows below are the honest signal: accuracy falls to 86.5% (JPEG), 49.5% (resize) and 62.0% (blur), which is why the product surfaces a per-upload robustness consistency score.

12. Robustness testing

Condition	Accuracy	F1
clean	100.00%	100.00%

Condition	Accuracy	F1
jpeg	86.50%	88.11%
resize	49.50%	49.75%
blur	62.00%	65.77%
noise	100.00%	100.00%

The UI robustness check replays the same idea per upload: JPEG, resize and blur variants are classified live and reported as e.g. 'Prediction consistency: 4/4'. Low consistency is presented as a caution signal, not as proof.

13. Installation and usage

Double-click run.bat (automatic: venv, dependencies, first-run training, server on http://localhost:5000); or use Docker (docker build -t verivision-ai . and docker run --rm -p 127.0.0.1:5000:5000 verivision-ai, or run-docker.bat); or follow the manual venv steps in HOW_TO_RUN.md (prepare_dataset → train_a1/a2/b/c → evaluate → run.py). Full detail, including prerequisites, download links and troubleshooting, is in HOW_TO_RUN.md.

14. Limitations

AI-based estimates only — not forensic proof; performance degrades on heavy compression, tiny images and generators unlike the training distribution; demo-data metrics do not transfer to real photography; CPU inference takes tens of seconds per image; gating thresholds are defaults, not tuned optima; the product gives one global verdict per image with no localization beyond the Grad-CAM overlay.

15. Future engineering improvements

Re-train on the full public CIFAKE corpus (or larger generator-diverse data) and re-tune ensemble weights and gating thresholds on a validation split; add calibration (temperature scaling) and precision-recall operating points; extend the robustness battery (WebP, crops, rotations); cache explanation variants; add optional batch-review mode; track dataset and checkpoint provenance in model_metadata.json for auditability.